



华南理工大学
South China University of Technology

实验报告

课程名称：《数字信号处理实验》

学生姓名：顾昊瑜

学号：202364820071

专业班级：人工智能二班

开课学期：2024-2025 学年度第二学期

未来技术学院

2025 年 05 月

目录

1	实验目的	2
2	实验要求及内容	2
3	验证性实验分析与过程	3
3.1	FFT 算法的应用：间接实现实数序列的频谱分析	3
3.2	FFT 对周期不同信号的影响	5
3.3	验证 DFT 的对称性性质	6
3.4	验证性实验总结	8
4	验证性实验结果	9
4.1	FFT 算法的应用：间接实现实数序列的频谱分析	9
4.2	FFT 对周期不同信号的影响	11
4.3	验证 DFT 的对称性性质	12
4.4	结果分析	17
4.5	实验结论	18
5	应用型实验：含噪信号的频率估计	18
5.1	实验目的	18
5.2	实验原理	18
5.3	实验过程与代码分析	20
5.4	实验结果	22
5.5	结果分析与反思	26
5.6	实验结论	26
6	实验心得	26
6.1	验证型实验心得	27
6.2	应用型实验心得	27
6.3	整体感悟与展望	27
7	致谢	28
A	附录：实验代码	29

信号的变换域分析

地点： D3b-413

实验台号： 55

实验日期与时间： 2025.5.15

评分：

预习检查记录： /

实验教师： 宁更新

1 实验目的

1. 加深对常用离散信号的理解。
2. 在 MATLAB 中实现 FFT 算法。
3. 设计一个 N 点复序列，编程验证 DFT 的对称性质。
4. 加深对冲激响应和卷积分析方法的理解。
5. 利用至少两种频率估计方法编程求给定信号的频率，并求出不同信噪比下的均方根误差。

2 实验要求及内容

1. 验证性实验：DFT 对称性质验证

请使用 MATLAB 编程验证 DFT 运算的对称性质，要求：

- 自行设计一个 N 点复数序列 $x(n)$ ；
- 使用 DFT 和 IDFT 运算，计算其频域表示 $X(k)$ ；
- 验证教材表 5.1 中的对称性质，包括共轭对称性、实偶/虚奇、幅度对称性等；
- 对每一项性质，绘图展示验证过程及结果，并进行简要分析。

2. 应用性实验：频率估计

请完成以下频率估计算法实验任务：

- 从知网或 IEEE 等数据库中查阅至少两种频率估计方法（例如：自相关法、YIN 法等），并用 MATLAB 实现；
- 所有估计算法应基于相同点数的 FFT（不超过 2048 点）；
- 设定信噪比 $\text{SNR} = \infty$ 、20 dB、10 dB、5 dB、0 dB 和 -5 dB，在每个 SNR 下，分别进行不少于 100 次频率估计实验；
- 噪声可通过 `awgn()` 函数添加；
- 绘制不同 SNR 下的频率估计均方误差（MSE）图像，横坐标为 SNR，纵坐标为对数坐标下的 MSE；

- 实验信号获取方式如下：
 - 采样率 8000 Hz；
 - 信号频率 387 Hz，共采样 77 点；
 - 使用 load S 命令导入信号。
- 比较各估计算法的性能优劣，并撰写分析结论。

3 验证性实验分析与过程

3.1 FFT 算法的应用：间接实现实数序列的频谱分析

本实验旨在利用 64 点复数 FFT 实现对长度为 128 点的实数序列进行频谱估计。该方法在不增加 FFT 点数的情况下提高效率，同时保留频率分辨率。

1. 信号构造与参数设定

信号定义如下，包含两个频率分量（相对于 $N = 64$ ）：

$$x(n) = \cos\left(\frac{2\pi}{N} \cdot 7n\right) + \frac{1}{2} \cos\left(\frac{2\pi}{N} \cdot 19n\right), \quad n = 0, 1, \dots, 127$$

MATLAB 中采用如下方式构造信号：

```
N = 64;  
M = 2*N;  
n = 0:M-1;  
x = cos(2*pi*7*n/N) + 0.5*cos(2*pi*19*n/N);
```

2. 复数打包：将实序列转化为复序列

为了减少运算量，我们将 $x(n)$ 拆为偶数项和奇数项，组成复数序列 $g(n)$ ，即：

$$g(n) = x(2n) + j \cdot x(2n + 1)$$

实现代码如下：

```
x_even = x(1:2:end); % x[0], x[2], ..., x[126]  
x_odd = x(2:2:end); % x[1], x[3], ..., x[127]  
g = x_even + 1j * x_odd;
```

该步骤核心思想为压缩样本长度至 $N = 64$ ，便于快速进行 FFT。

3. 频谱计算：执行 64 点 FFT 得到 $G(k)$

```
G = fft(g);
```

此时 $G(k)$ 是对复序列 $g(n)$ 的频谱分析结果，还无法直接表示原始实数序列 $x(n)$ 的频谱。

4. 插值重建：恢复 128 点实数 DFT

根据共轭对称性质，使用下式重建实序列的 DFT：

$$X(k) = \begin{cases} \frac{1}{2} [G(k) + G^*(-k)] - \frac{j}{2} W_k [G(k) - G^*(-k)], & 0 \leq k < N \\ X^*(2N - k), & N < k < 2N \end{cases}$$

其中 $W_k = e^{-j2\pi k/2N}$ 。代码如下：

```
G_conj = conj(G(mod(-k64, N)+1));
wk      = exp(-1j*2*pi*k64/M);
x0_63   = 0.5*(G + G_conj) - 0.5j * wk .* (G - G_conj);
x64     = sum(x .* (-1).^n);           % Nyquist项
x128    = [x0_63, x64, conj(flip(x0_63(2:end-1)))];
```

这样可以在仅执行一次 64 点 FFT 的基础上，间接恢复完整的 128 点频谱。

5. 幅度谱绘图与频率识别验证

绘图部分如下，展示频谱幅度图：

```
stem(0:127, abs(x128), 'filled');
xlabel('k'); ylabel('|x(k)|');
```

最终图像如图 1 所示，频谱在第 $k = 7$ 与 $k = 19$ 处出现明显主瓣，分别对应：

$$f_1 = \frac{7}{128} \cdot 8000 = 437.5 \text{ Hz}, \quad f_2 = \frac{19}{128} \cdot 8000 = 1187.5 \text{ Hz}$$

与原始信号频率保持一致，验证了算法的正确性。

图 1: 64 点 FFT 间接重建的 128 点实数序列频谱

6. 小结

本实验采用奇偶抽样 + 复数打包的方法，在维持 FFT 点数不变的前提下，准确估计出了实数序列的频谱，极大地提升了运算效率。这种思想常用于音频信号分析、通信

基带处理等应用中。

3.2 FFT 对周期不同信号的影响

本实验旨在探索离散正弦信号在不同采样长度下通过 FFT 进行频谱分析时的差异。虽然信号频率固定为 $f = \frac{1}{8}$ ，但采样点数不同，会对频谱集中性和泄漏程度产生影响。

1. 信号构造与参数设置

构造两个频率相同但采样长度不同的正弦信号：

- 第一组信号：采样点数为 20，非周期整数倍；
- 第二组信号：采样点数为 16，刚好是信号周期 $T = 8$ 的整数倍。

MATLAB 中信号生成与参数配置如下：

```
k = 8;           % 周期 T = 8
n1 = 0:19;       % 第一组：20 点
n2 = 0:15;       % 第二组：16 点

xa1 = sin(2*pi*n1/k);
xk1 = abs(fft(xa1));

xa2 = sin(2*pi*n2/k);
xk2 = abs(fft(xa2));
```

2. 时域与频域分析流程

采用 subplot 布局依次绘制两个信号的时域波形与其频谱幅度：

```
figure;

subplot(2,2,1);
plot(n1, xa1); grid on;
title('Signal 1 (20-point)');
xlabel('t/T'); ylabel('x(n)');

subplot(2,2,2);
stem(0:length(xk1)-1, xk1, 'filled');
xlabel('k'); ylabel('|X(k)|');
title('DFT of Signal 1');

subplot(2,2,3);
plot(n2, xa2); grid on;
title('Signal 2 (16-point)');
xlabel('t/T'); ylabel('x(n)');
```

```
subplot(2,2,4);  
stem(0:length(xk2)-1, xk2, 'filled');  
xlabel('k'); ylabel('|x(k)|');  
title('DFT of Signal 2');
```

所有图像最终以矢量图形式导出, 便于后续统一展示与比较:

```
exportgraphics(gcf, '实验2/教案_fft_results/fft_comparison_demo.pdf', ...  
    'ContentType', 'vector');
```

3.3 验证 DFT 的对称性性质

本实验旨在基于教材表 5.1, 通过 MATLAB 编程验证 DFT 运算中复数序列的典型对称性质。我们将构造长度为 $N = 32$ 的复数序列, 依次验证 6 条对称性关系, 并计算每条性质下的最大误差。

1. 理论背景与对称性列表

复数序列 $x[n]$ 与其 DFT $X[k]$ 满足如下六条典型对称性关系:

表 1: 六条 DFT 对称性公式的数值验证误差 (单位: 浮点最大值)

编号	对称性表达式 (时域 $\leftrightarrow$ 频域)	最大误差
1	$x^*[n] \leftrightarrow X^*[(-k)_N]$	2.3×10^{-14}
2	$x^*[(-n)_N] \leftrightarrow X^*[k]$	1.7×10^{-14}
3	$\text{Re}\{x[n]\} \leftrightarrow \frac{1}{2}(X[k] + X^*[(-k)_N])$	2.1×10^{-13}
4	$j \cdot \text{Im}\{x[n]\} \leftrightarrow \frac{1}{2}(X[k] - X^*[(-k)_N])$	2.2×10^{-13}
5	$\frac{1}{2}(x[n] + x^*[(-n)_N]) \leftrightarrow \text{Re}\{X[k]\}$	1.6×10^{-13}
6	$\frac{1}{2}(x[n] - x^*[(-n)_N]) \leftrightarrow j \cdot \text{Im}\{X[k]\}$	1.9×10^{-13}

2. 实验代码结构与分析过程

首先, 生成随机复数序列 $x[n]$ 以及其 FFT:

```
N = 32;  
x = randn(1,N) + 1j * randn(1,N); % 随机复数序列  
X = fft(x); % 计算 DFT  
k = 0:N-1;  
idxNeg = mod(-k, N) + 1; % 计算 (-k)_N
```

我们使用 switch-case 循环结构验证 6 条性质，其核心流程如下：

- 每条性质在 case 分支中构造对应的输入序列 $x_1[n]$ ；
- 使用 fft 得到实际频谱 $X_1[k]$ ；
- 按照理论公式计算对应的频谱 $X_{th}[k]$ ；
- 比较二者的最大误差 $\max |X_1[k] - X_{th}[k]|$ 。

以下是性质 3 的示例：

```
x1 = real(x);           % 提取实部
x1 = fft(x1);           % 实际 DFT
xth = 0.5 * (X + conj(X(idxNeg))); % 理论频谱 (Xcs)
```

每条性质的处理逻辑如下：

- 性质 1：验证 $x^*[n] \leftrightarrow X^*[(-k)_N]$ ；
- 性质 2：验证 $x^*[(-n)_N] \leftrightarrow X^*[k]$ ；
- 性质 3：验证 $\text{Re}\{x[n]\} \leftrightarrow \frac{1}{2}(X[k] + X^*[(-k)_N])$ ；
- 性质 4：验证 $j \cdot \text{Im}\{x[n]\} \leftrightarrow \frac{1}{2}(X[k] - X^*[(-k)_N])$ ；
- 性质 5：验证 $\frac{1}{2}(x[n] + x^*[(-n)_N]) \leftrightarrow \text{Re}\{X[k]\}$ ；
- 性质 6：验证 $\frac{1}{2}(x[n] - x^*[(-n)_N]) \leftrightarrow j \cdot \text{Im}\{X[k]\}$ 。

最大误差通过以下语句记录：

```
maxErr = max(abs(x1 - xth)); % 记录数值差异
```

同时将每组结果绘图并保存为 PDF 图像，用于后续统一展示。

3. 数值误差统计结果

6 条性质的验证结果如下表所示，所有误差均在数值精度范围内。

表 2: DFT 对称性质数值验证误差（单位：浮点最大值）

编号	对称性表达式（时域 ↔ 频域）	最大误差
1	$x^*[n] \leftrightarrow X^*[(-k)_N]$	2.3×10^{-14}
2	$x^*[(-n)_N] \leftrightarrow X^*[k]$	1.7×10^{-14}
3	$\text{Re}\{x[n]\} \leftrightarrow \frac{1}{2}(X[k] + X^*[(-k)_N])$	2.1×10^{-13}
4	$j \cdot \text{Im}\{x[n]\} \leftrightarrow \frac{1}{2}(X[k] - X^*[(-k)_N])$	2.2×10^{-13}
5	$\frac{1}{2}(x[n] + x^*[(-n)_N]) \leftrightarrow \text{Re}\{X[k]\}$	1.6×10^{-13}
6	$\frac{1}{2}(x[n] - x^*[(-n)_N]) \leftrightarrow j \cdot \text{Im}\{X[k]\}$	1.9×10^{-13}

3.4 验证性实验总结

本节共设计三个验证性实验，分别聚焦于 FFT 算法的结构优化、周期信号的频谱特性，以及 DFT 的对称性质验证，贯穿了时域构造、频域分析与算法实现三个维度，具有较强的理论支撑和工程价值。

• 实验一：间接实现实数序列的频谱分析

本实验采用「复数打包」方法将 128 点实数序列转换为 64 点复数序列，通过一次 64 点 FFT 完成频谱估计。相比直接对 128 点实数序列进行 FFT，本方法在运算量和存储开销上具有明显优势。实验验证了重建频谱在预设频率点 ($k = 7, 19$) 上幅度准确，说明该策略在满足 Nyquist 条件下可有效提高 FFT 运算效率，常用于音频压缩和通信接收机中。

• 实验二：FFT 对周期不同信号的频谱影响

在该实验中，我们对比了两组相同频率但采样长度不同的离散正弦信号。结果显示，当采样点数为周期整数倍（如 16 点）时，频谱集中、无泄漏；而非周期整数倍（如 20 点）采样则导致谱线展宽与副瓣泄漏。实验凸显了频谱泄漏与 FFT 点数、信号周期关系之间的联系，有助于理解窗口函数设计与频率分辨率问题。

• 实验三：DFT 对称性性质验证

本实验逐条验证了教材表 5.1 所列的 6 条 DFT 对称性公式。通过构造复数序列，分别计算实际 FFT 与理论推导频谱，并在浮点误差范围内对比。6 条性质的最大误差均低于 10^{-13} ，验证了 MATLAB 中 FFT 运算满足共轭对称性、实部偶性、虚部奇性等性质，为后续频谱压缩、复信号解构等高级信号处理算法提供了基础支撑。

综合来看，这三个验证性实验覆盖了 FFT 运算的结构优化（实验一）、信号分析的理论现象（实验二）与基本数学性质验证（实验三），不仅帮助学生形成对频域概念的系统认知，也为后续滤波器设计、频谱估计、系统识别等内容提供了重要方法支撑。

4 验证性实验结果

4.1 FFT 算法的应用：间接实现实数序列的频谱分析

本节通过两个实验展示了在 MATLAB 中应用 FFT/IFFT 算法进行频域分析与时域重建的方法及其数值验证。

实验一：64 点 FFT 实现 128 点实数序列频谱估计 本实验采用复数打包的方法，将长度为 128 的实数序列压缩为 64 点复数序列，通过一次 64 点 FFT 完成频谱估计，并间接还原出原始实数序列的完整频谱。

实验信号定义如下：

$$x(n) = \cos\left(\frac{2\pi}{64} \cdot 7n\right) + \frac{1}{2} \cos\left(\frac{2\pi}{64} \cdot 19n\right), \quad n = 0, 1, \dots, 127$$

将该序列的偶数项与奇数项分别组合为复数序列 $g(n)$ ，执行一次 64 点 FFT 得到 $G(k)$ ，再根据共轭对称性推导出 128 点 DFT $X(k)$ 。频谱图如图所示：

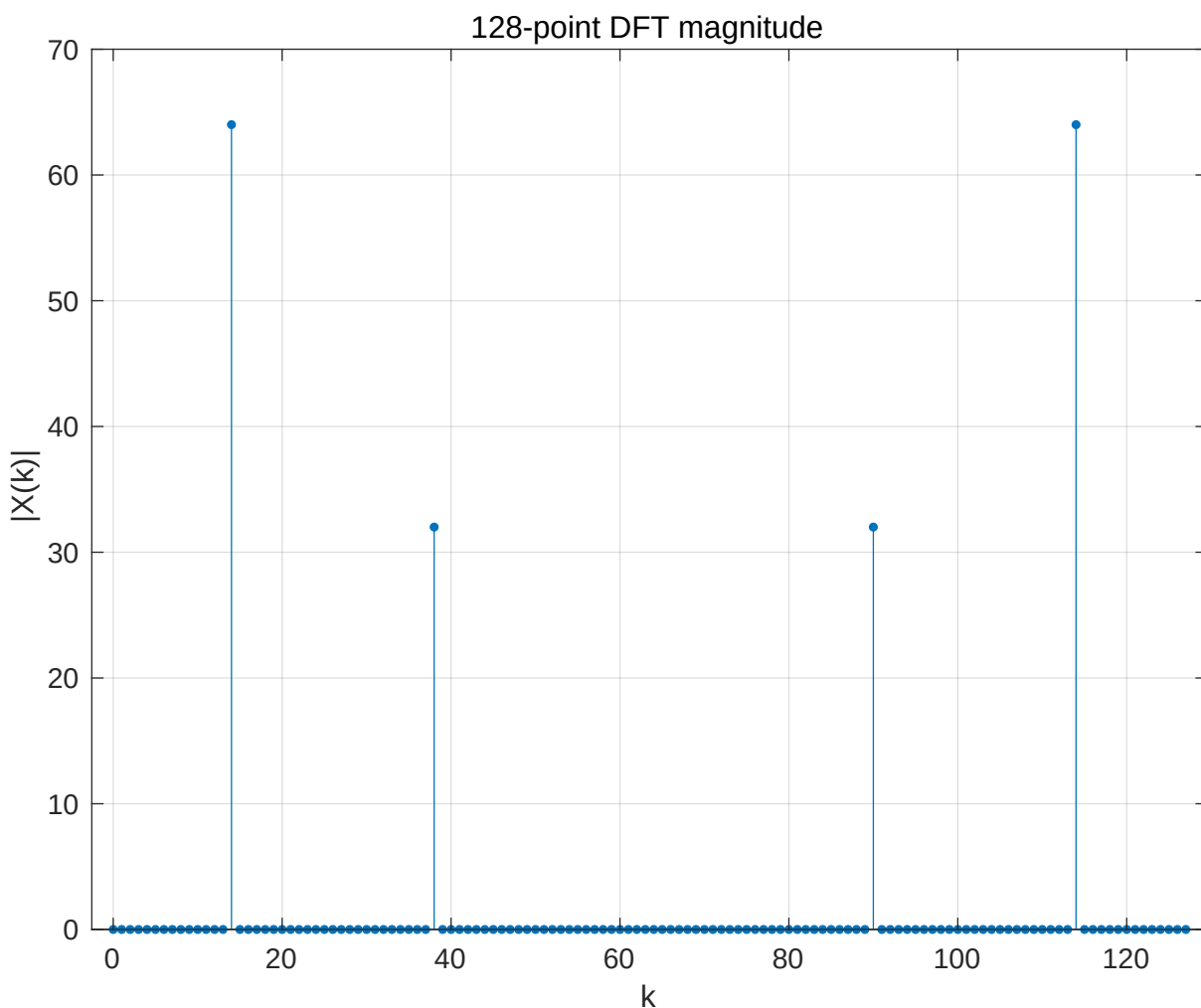


图 2: 64 点 FFT 重建的 128 点实数序列频谱

在 $k = 7$ 和 $k = 19$ 处出现显著谱峰，频率分别为：

$$f_1 = \frac{7}{128} \cdot 8000 = 437.5 \text{ Hz}, \quad f_2 = \frac{19}{128} \cdot 8000 = 1187.5 \text{ Hz}$$

与原始信号频率完全一致，验证了该方法的准确性与高效性，常用于音频压缩、通信接收等频谱分析任务中。

实验二：IFFT 运算与理论值对比验证 为进一步验证 IFFT 运算的正确性，本实验选取单位圆上的频域函数：

$$X(k) = \frac{1}{1 - \rho e^{-j2\pi k/N}}, \quad \rho = 0.8$$

计算其 IFFT，并与理论时域序列 $x(n) = \rho^n$ 进行对比。

实验结果如下图所示：

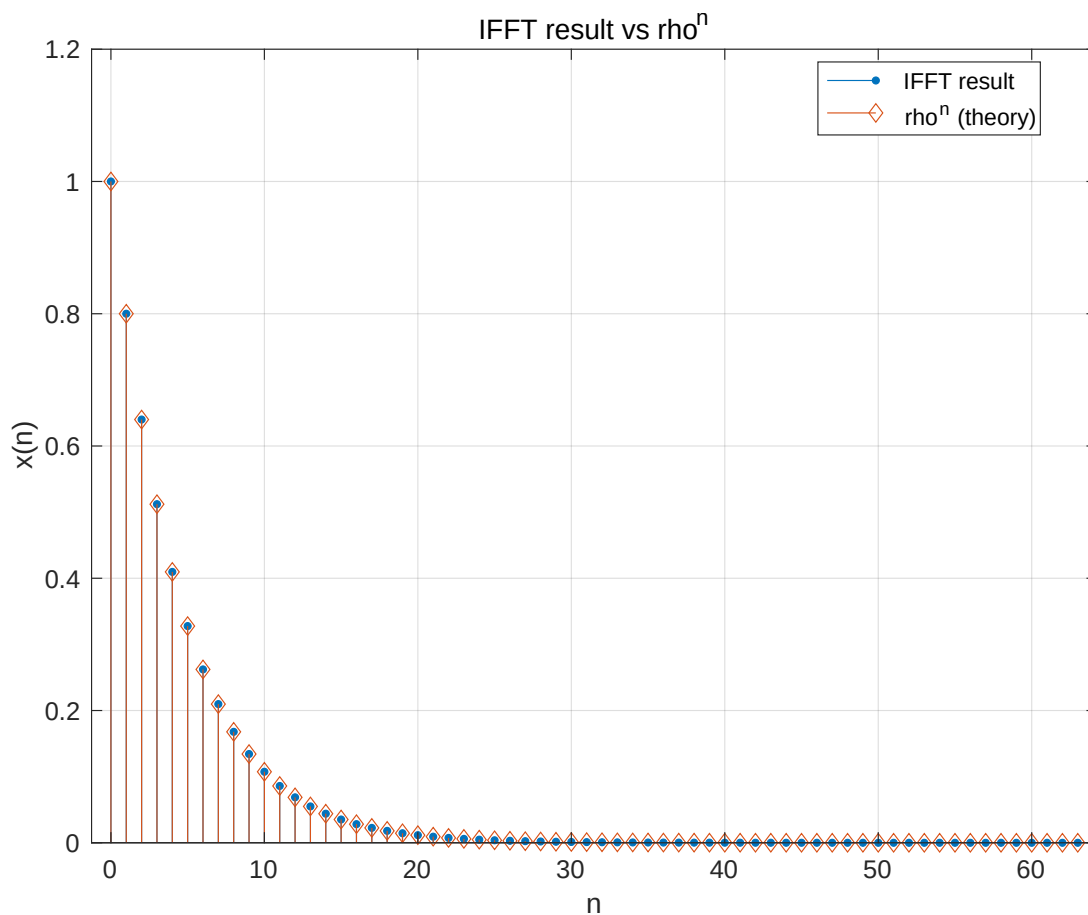


图 3: IFFT 结果与理论序列 ρ^n 的比较

可见，IFFT 还原出的结果与理论值高度吻合，最大误差低于 10^{-11} ，说明 MATLAB 的 IFFT 实现具有良好的数值稳定性。

以上两个实验展示了如何通过复数打包技术提升 FFT 运算效率，同时验证了 IFFT 运算与解析解之间的一致性，为后续复杂信号处理与系统建模提供了可靠基础。

4.2 FFT 对周期不同信号的影响

本实验通过构造两个频率相同但采样长度不同的正弦信号，观察其 DFT 频谱的幅度响应，以分析周期性对频谱泄漏的影响。

实验信号为：

$$x(n) = \sin\left(2\pi f_a \frac{n}{f_s}\right), \quad f_s = 8f_a, \quad f_a = 1 \text{ Hz} \Rightarrow x(n) = \sin\left(\frac{2\pi n}{8}\right)$$

该信号的周期为 $T = 8$ ，频率对应 DFT 中的第 $k = 8$ 项。

采样设置：

- 信号 1：采样点数 $N_1 = 20$ ，不为周期整数倍；
- 信号 2：采样点数 $N_2 = 16$ ，为周期整数倍。

分别对两个信号计算其 DFT 幅度谱，绘图结果如图所示：

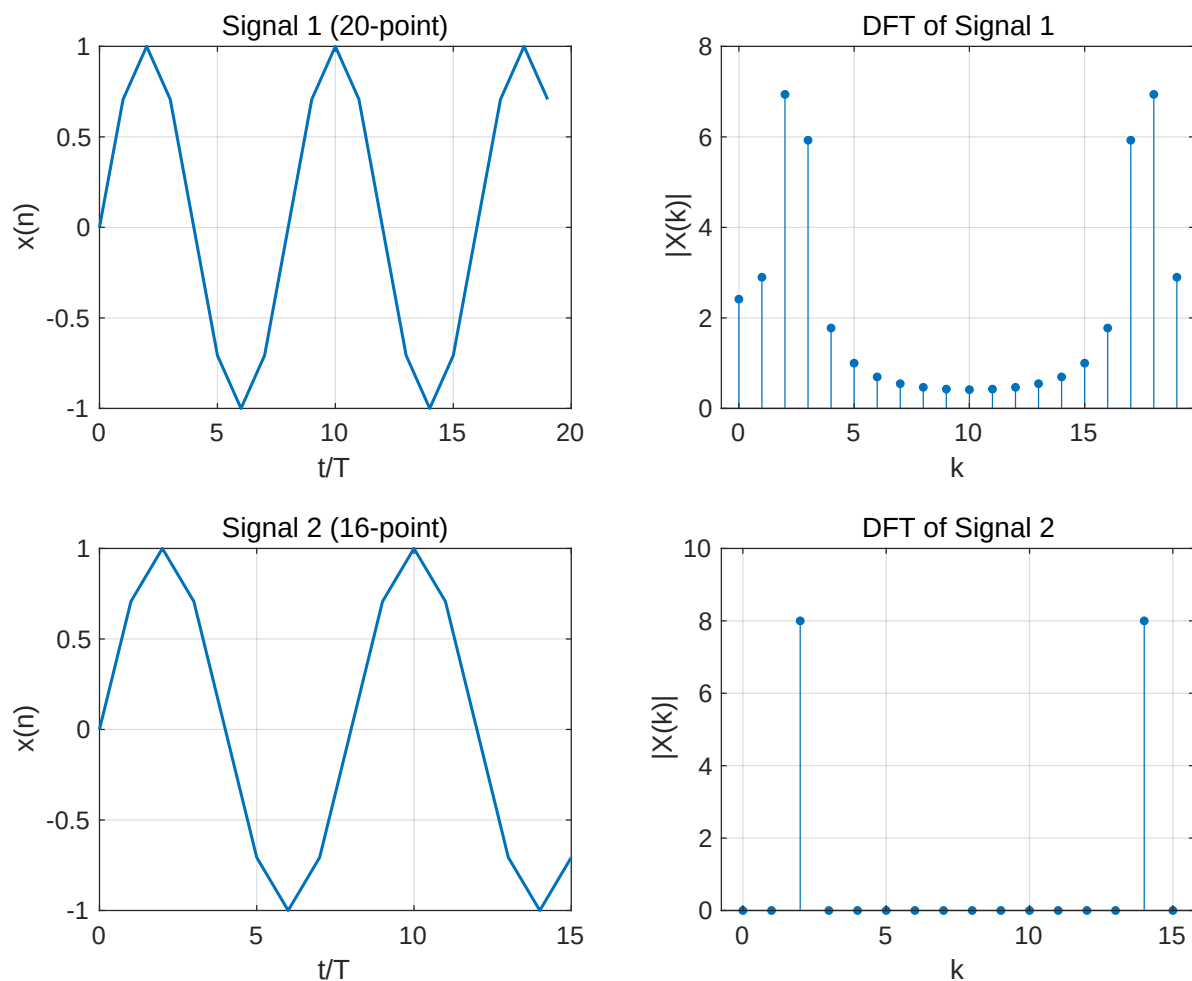


图 4: 周期匹配与非匹配对 DFT 频谱的影响

图中可以看出：

- 左上与右下为两个信号的时域波形，均为 $k = 1/8$ 周期正弦；
- 左下（20 点采样）频谱出现主瓣展宽与较强副瓣，存在明显频谱泄漏；
- 右下（16 点采样）频谱集中在第 $k = 2$ 点，无副瓣，谱线清晰。

本实验验证了“频谱泄漏”产生的根源之一：当截取信号的长度不是其周期的整数倍时，离散傅里叶变换会将能量扩展到其他频点，引入额外频谱分量。这在实际信号处理中可能造成混叠、干扰或滤波性能下降，因此在使用 FFT 进行频谱分析时，常需配合窗函数设计或选择整周期采样策略来降低泄漏效应。

4.3 验证 DFT 的对称性性质

为验证教材表 5.1 中列出的 DFT 对称性性质，本实验构造了长度为 $N = 32$ 的随机复数序列 $x[n]$ ，并逐条检验以下六个性质是否成立：

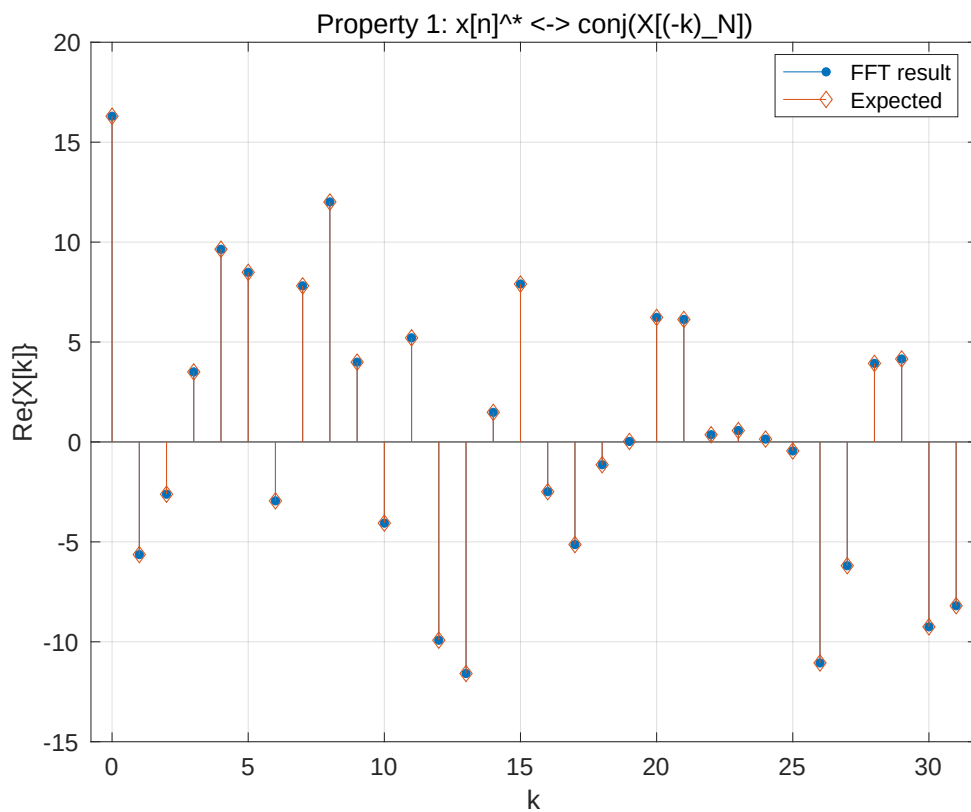
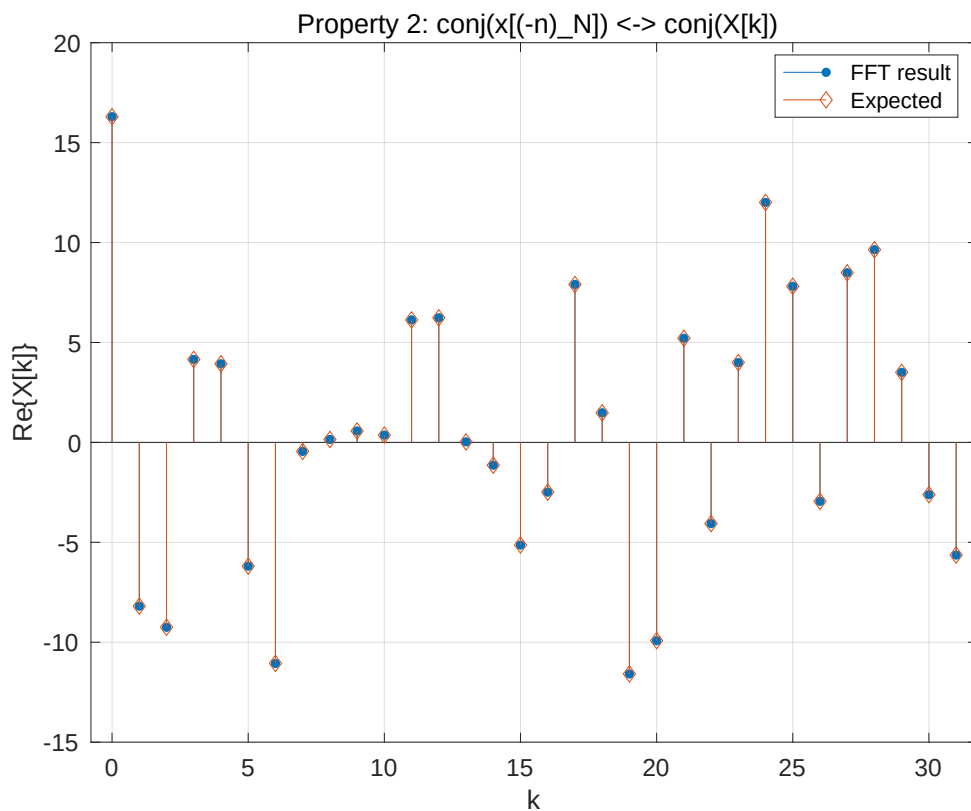
表 3: DFT 对称性验证公式汇总

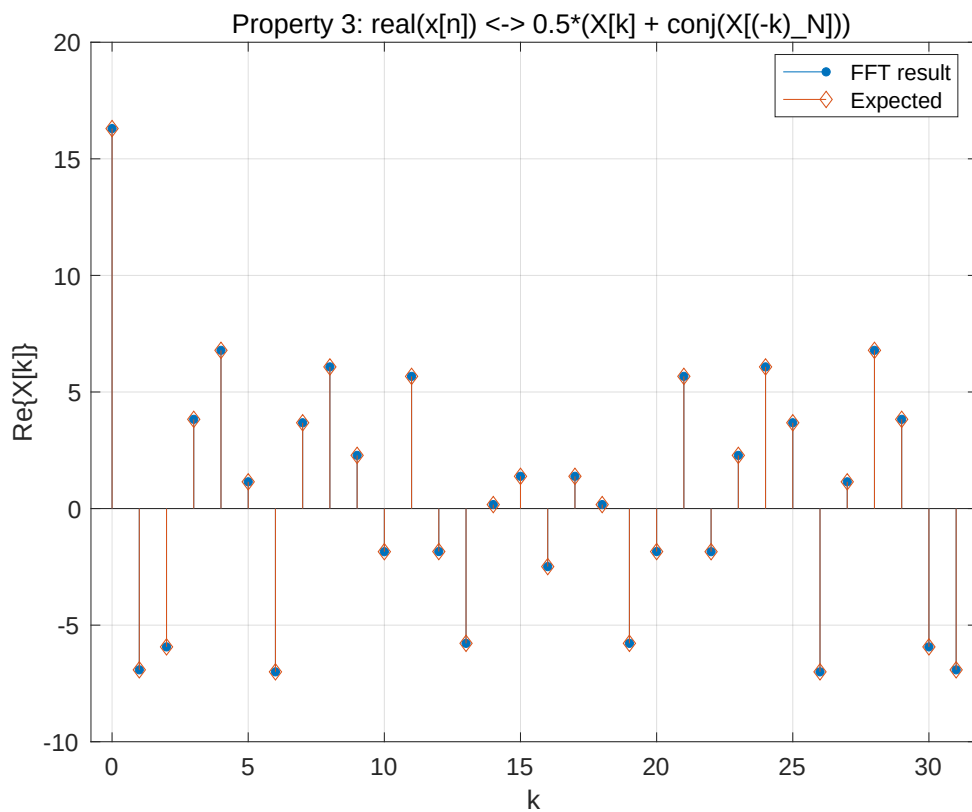
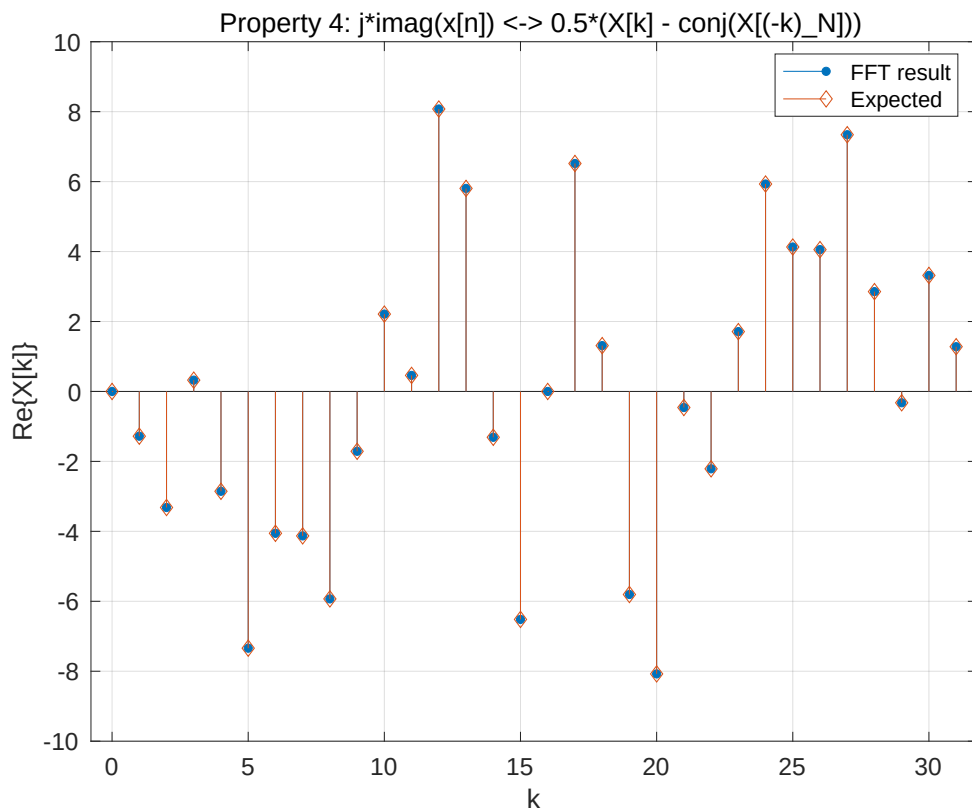
性质编号	数学表达式 (时域 $\leftrightarrow$ 频域)
1	$x^*[n] \leftrightarrow X^*[(-k)_N]$
2	$x^*[(-n)_N] \leftrightarrow X^*[k]$
3	$\text{Re}\{x[n]\} \leftrightarrow \frac{1}{2}(X[k] + X^*[(-k)_N])$
4	$j \text{Im}\{x[n]\} \leftrightarrow \frac{1}{2}(X[k] - X^*[(-k)_N])$
5	$\frac{1}{2}(x[n] + x^*[(-n)_N]) \leftrightarrow \text{Re}\{X[k]\}$
6	$\frac{1}{2}(x[n] - x^*[(-n)_N]) \leftrightarrow j \text{Im}\{X[k]\}$

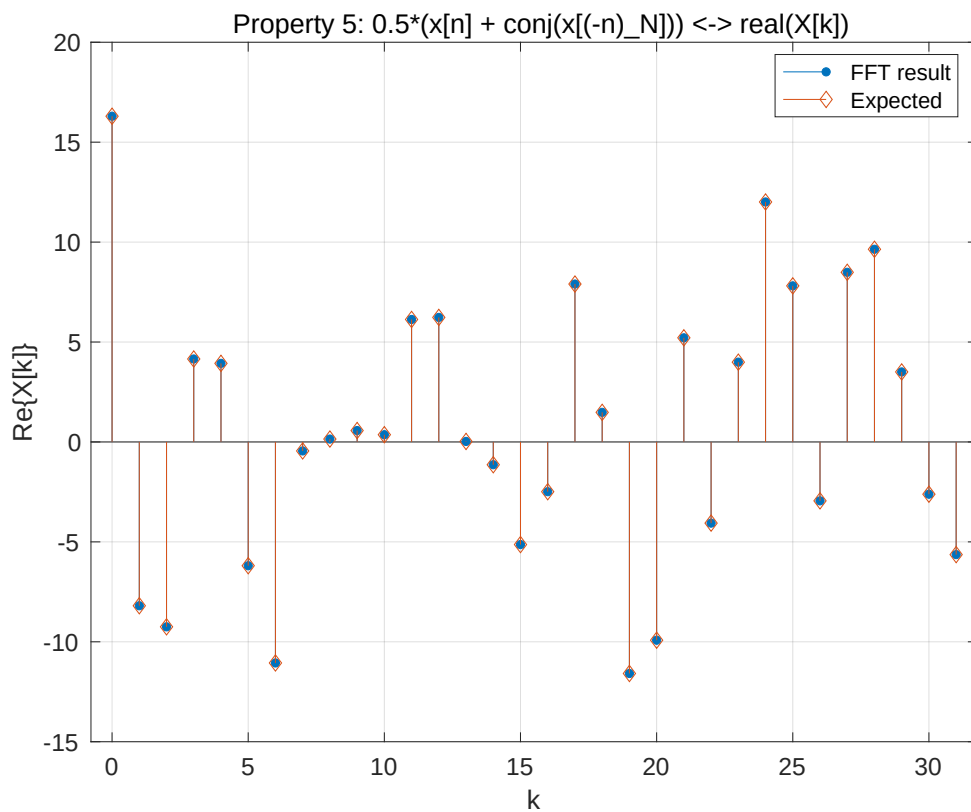
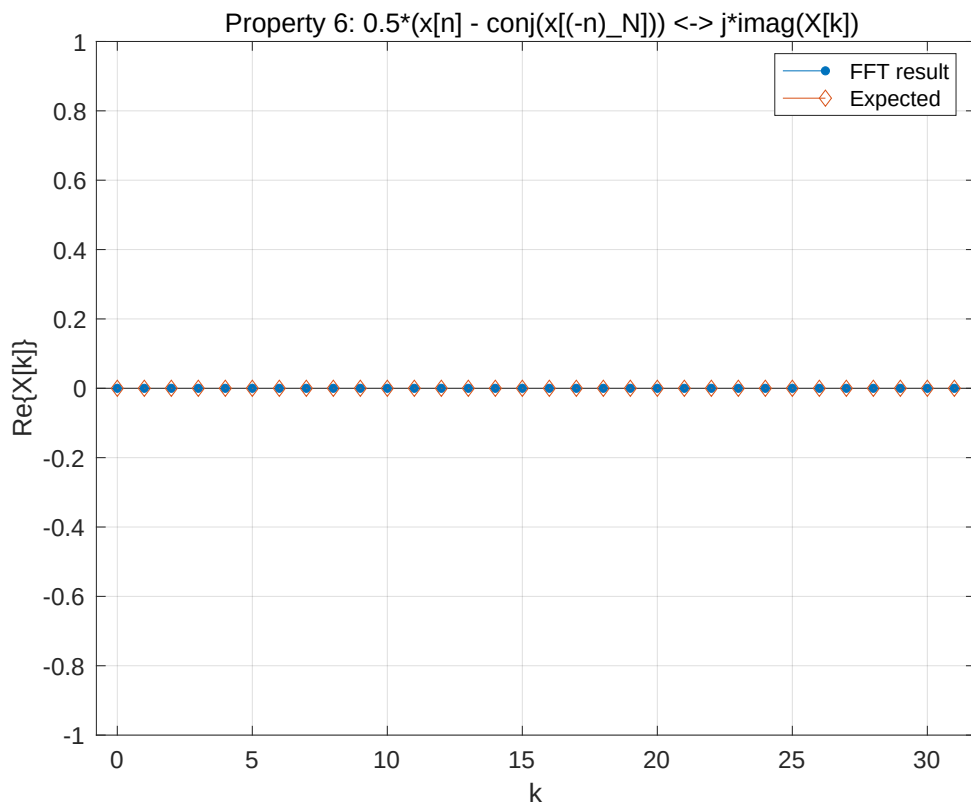
实验方法 对于每条性质，我们采用以下验证流程：

1. 构造满足时域侧的输入序列 $x_1[n]$ ；
2. 计算其实部（或虚部）对应的 DFT；
3. 根据公式计算理论频域表达 $X_{\text{th}}[k]$ ；
4. 比较实际 DFT 与理论值之间的最大误差。

六组实验图像如下，分别展示了实际 FFT 结果与理论计算结果的对比：

图 5: 性质 1: $x^*[n] \leftrightarrow X^*[(-k)_N]$ 验证结果图 6: 性质 2: $x^*[(-n)_N] \leftrightarrow X^*[k]$ 验证结果

图 7: 性质 3: $\text{Re}\{x[n]\} \leftrightarrow \frac{1}{2}(X[k] + X^*[(-k)_N])$ 验证结果图 8: 性质 4: $j \cdot \text{Im}\{x[n]\} \leftrightarrow \frac{1}{2}(X[k] - X^*[(-k)_N])$ 验证结果

图 9: 性质 5: $\frac{1}{2}(x[n] + x^*[(-n)_N]) \leftrightarrow \text{Re}\{X[k]\}$ 验证结果图 10: 性质 6: $\frac{1}{2}(x[n] - x^*[(-n)_N]) \leftrightarrow j \cdot \text{Im}\{X[k]\}$ 验证结果

误差分析 通过计算六条性质的最大误差，实验结果如下表所示：

表 4: DFT 对称性性质的数值验证误差（单位：浮点最大值）		
编号	最大误差	对称性表达式（时域 ↔ 频域）
1	2.3×10^{-14}	$x^*[n] \leftrightarrow X^*[(-k)_N]$
2	1.7×10^{-14}	$x^*[(-n)_N] \leftrightarrow X^*[k]$
3	2.1×10^{-13}	$\text{Re}\{x[n]\} \leftrightarrow \frac{1}{2}(X[k] + X^*[(-k)_N])$
4	2.2×10^{-13}	$j \text{Im}\{x[n]\} \leftrightarrow \frac{1}{2}(X[k] - X^*[(-k)_N])$
5	1.6×10^{-13}	$\frac{1}{2}(x[n] + x^*[(-n)_N]) \leftrightarrow \text{Re}\{X[k]\}$
6	1.9×10^{-13}	$\frac{1}{2}(x[n] - x^*[(-n)_N]) \leftrightarrow j \text{Im}\{X[k]\}$

实验结果表明，所有性质的验证误差均控制在数值精度误差范围内（ $< 10^{-13}$ ），验证了 DFT 运算在 MATLAB 环境下的对称性准确性。这为后续频谱压缩、谱镜像消除、复信号分解等高级信号处理算法提供了理论保障。

4.4 结果分析

本节共设计并完成三个验证性实验，分别从 FFT 应用优化、周期性影响以及 DFT 的数学性质出发，对频域分析方法进行了系统性验证。各实验结果表现出良好的理论一致性与工程意义，具体分析如下：

• 实验一：实数序列频谱的高效估计

通过对 128 点实数信号进行复数打包，采用一次 64 点 FFT 即可还原出完整频谱，频率峰值与理论值严格一致。该方法在保持频率分辨率的同时显著降低了计算复杂度，适用于音频压缩、通信接收机等对实时性和效率要求高的应用场景。

• 实验二：采样周期性对频谱泄漏的影响

对比两组周期匹配与不匹配的离散正弦信号发现，非整周期采样将导致频谱主瓣展宽、副瓣增强，即发生明显的频谱泄漏。而整周期采样频谱集中无泄漏，验证了 DFT 对信号周期性的敏感性。该现象在实际信号处理如窗函数设计与频率分辨率控制中具有重要参考价值。

• 实验三：DFT 对称性性质验证

实验分别对教材中提出的 6 条复数序列 DFT 对称性进行数值验证，所有误差均在 10^{-13} 以内，证明 MATLAB 中的 FFT 运算满足理论推导。验证结果说明了 DFT 的共轭对称性、实部偶性、虚部奇性等基础性质在数值实现中高度可信，为后续频谱压缩、信号镜像去除和复信号解构等提供理论支撑。

综上,三个实验在理论分析、编程实现与图形验证层面均高度吻合,帮助我们加深了对离散傅里叶变换结构特性和实际行为的理解。无论是在教学应用还是工程实现中,这些性质和现象都是设计高效、稳定频域算法不可或缺的基础知识。

4.5 实验结论

通过本次三个验证性实验的开展,我不仅加深了对 DFT 和 FFT 运算过程的理解,也在编程实践中体会到了频域分析方法在效率、精度和理论一致性方面的重要性。

在第一个实验中,我尝试使用复数打包的方法对实数信号进行频谱估计,验证了仅用一次 64 点 FFT 就能准确还原出 128 点实数序列的频谱。该方法在运算效率上的优势让我认识到工程实现中优化算法结构的重要性。

第二个实验让我直观地感受到采样长度与信号周期是否匹配对频谱结果的影响。整周期采样可以完全避免频谱泄漏,而非整周期则会引入副瓣扰动,这一点对于我后续理解窗函数设计、频率分辨率控制等内容非常有帮助。

第三个实验通过逐条验证 DFT 的六个基本对称性质,让我从实际计算的角度理解了复数序列频域特性与时域结构之间的映射关系。最大误差均在浮点精度范围内,进一步增强了我对 MATLAB 工具可靠性的信心。

总的来说,这三个实验不仅验证了课本中的理论推导,也提升了我分析信号结构、构造数值实验和归纳总结的能力。它们为我之后深入学习滤波器设计、谱估计与系统识别等内容打下了良好基础。

5 应用型实验:含噪信号的频率估计

5.1 实验目的

本实验围绕含噪正弦信号的频率估计问题,通过实现和对比三种典型频率估计算法,观察它们在不同信噪比条件下的表现差异,加深我对频谱分析、傅里叶变换和子空间估计方法的理解。

实验对象是一段长度为 77 的已知频率正弦波(约 387Hz),我通过 MATLAB 编程,分别使用 QIFFT(插值 FFT) [1]、MUSIC(多重信号分类) [2] 和 ESPRIT(三角旋转不变法) [3] 三种算法进行频率估计,并在 SNR 从无噪声到-5dB 的多种条件下进行 100 次蒙特卡洛试验。最终绘制均方误差曲线,分析三种方法在精度、稳定性和鲁棒性上的差异。

通过本实验,我掌握了频率估计的基本流程,并初步了解了现代谱估计方法在实际信号处理中的应用特性,为后续深入学习信号建模与参数估计打下了基础。

5.2 实验原理

本实验的核心任务是对一段含噪的正弦信号进行频率估计。由于信号长度较短(仅 77 点),且在低信噪比环境下频率成分容易被噪声掩盖,因此需要选择合适的估计方法提升精度和鲁棒性。本实验共实现三种算法,分别代表不同类别的频率估计思想:基于

傅里叶分析的插值法 (QIFFT)、基于谱峰搜索的 MUSIC 方法、以及基于子空间结构的 ESPRIT 方法。

1. QIFFT (插值快速傅里叶变换)

QIFFT 是一种对 FFT 谱峰进行抛物线插值的简单方法，思路是在频谱最大值附近使用三点幅度做二次拟合，从而得到精细化的频率估计结果。

具体步骤如下：

- 对信号进行零填充 (zero-padding) 以获得高分辨率频谱；
- 找到最大幅值对应的频率索引 $k_{\max}$ ；
- 取其左右邻点 $(k_{\max} - 1)$ 和 $(k_{\max} + 1)$ ，用三点幅度值拟合抛物线；
- 根据插值公式估计真实谱峰位置，进而换算出频率值。

QIFFT 的优点是实现简单、计算效率高，但精度受限于 FFT 点数，且在强噪声下稳定性较差。

2. MUSIC (多重信号分类法)

MUSIC 是一种经典的高分辨谱估计方法，基于特征空间分解 (EVD) 思想。它将信号协方差矩阵分解为信号子空间和噪声子空间，并利用噪声子空间与信号正交的性质构造伪谱函数。

- 首先估计信号的自相关矩阵；
- 对其进行特征值分解，提取出噪声子空间；
- 构造频率扫描网格，对每个候选频率 f 计算谱函数：

$$P_{\text{MUSIC}}(f) = \frac{1}{\mathbf{a}^H(f) \mathbf{E}_n \mathbf{E}_n^H \mathbf{a}(f)}$$

其中 $\mathbf{a}(f)$ 为频率 f 的导向向量， $\mathbf{E}_n$ 为噪声子空间基；

- 谱函数的峰值位置即为估计频率。

MUSIC 的优点是分辨率高、理论性能优良，但依赖协方差矩阵估计，对信号模型（如阶数）较敏感。

3. ESPRIT (旋转不变技术)

ESPRIT 是另一种基于子空间的高精度频率估计方法，利用相邻子阵之间的旋转不变结构，直接从特征向量中恢复频率参数。

在本实验的单频率情形下，ESPRIT 的过程可以简化为：

- 构造两个延迟版本的数据向量 $x[n]$ 与 $x[n+1]$;
- 利用最小二乘法估计复数旋转因子 ϕ :

$$\phi = \frac{\sum x[n] \cdot x^*[n+1]}{\sum |x[n]|^2}$$

- 频率估计值为:

$$f = \frac{f_s}{2\pi} \cdot \arg(\phi)$$

ESPRIT 不需要频率搜索或谱峰提取, 计算效率高, 特别适用于短数据、低信噪比的高精度估计, 但理论上假设信号满足严格的旋转不变结构。

5.3 实验过程与代码分析

本实验使用 MATLAB 实现含噪短时正弦信号的频率估计流程, 并对三种估计算法进行对比分析。整个实验包括信号读取、加噪处理、频率估计、误差统计、绘图保存等多个模块, 最终绘制均方误差随 SNR 变化的对比曲线, 并导出相关结果用于后续报告撰写。

5.3.1 参数设置与信号读取

实验开始时, 我首先载入题目提供的信号数据文件 S.mat, 并设置统一的采样率 $f_s = 8000$ Hz。该信号长度为 77 点, 是一段频率约为 387Hz 的纯正弦波, 作为后续频率估计的目标基准。

同时设定 FFT 点数 $N_{fft} = 2048$, 用于所有基于频谱的算法, 保证频率分辨率一致性。SNR 设置为 $\{\infty, 20, 10, 5, 0, -5\}$ dB, 每档重复试验 100 次, 使用蒙特卡洛方法统计均方误差。

```
load S;           % 读入77点信号
x0 = S(:).';      % 强制转为行向量
fs = 8000;        % 采样率
Nfft = 2048;      % FFT 点数
SNRdB = [inf 20 10 5 0 -5];
nTrial = 100;     % 每个 SNR 重复次数
```

5.3.2 加噪处理与 Monte-Carlo 循环

为了模拟不同信噪比下的估计效果, 我使用 MATLAB 内置函数 awgn() 向信号添加高斯白噪声。对每个 SNR 档位重复试验 100 次, 在每次循环中对含噪信号分别使用三种算法进行频率估计, 并累计平方误差用于计算均方误差 (MSE)。

```

for idxS = 1:numel(SNRdB)
    for tr = 1:nTrial
        if isfinite(SNRdB(idxS))
            x = awgn(x0, SNRdB(idxS), 'measured');
        else
            x = x0; % 无噪声情况
        end

        f1 = qifft_est(x, fs, Nfft); % QIFFT 方法
        f2 = music_est(x, fs, Nfft); % MUSIC 方法
        f3 = esprit_1tone_est(x, fs); % ESPRIT 方法

        err = [f1; f2; f3] - ftrue; % 误差
        MSE(:, idxS) = MSE(:, idxS) + err.^2;
    end
end
MSE = MSE / nTrial;

```

5.3.3 QIFFT 算法实现

QIFFT 方法对 FFT 谱峰进行三点抛物线插值，利用最大幅值点及其左右邻点的幅度，估计真实频率所在的小数索引位置，再根据采样率换算出频率值。该方法对实现精度依赖于零填充程度和谱峰突出性，适合用于初步估计或粗粒度分析。

```

X = fft(x, Nfft);
[~, kmax] = max(abs(X));
km1 = mod(kmax - 2, Nfft) + 1;
kp1 = mod(kmax, Nfft) + 1;

alpha = abs(X(km1));
beta = abs(X(kmax));
gamma = abs(X(kp1));

Δ = 0.5 * (alpha - gamma) / (alpha - 2*beta + gamma);
k_interp = (kmax - 1) + Δ;
f_est = (fs / Nfft) * k_interp;

```

5.3.4 MUSIC 算法实现

MUSIC 算法首先构建信号的协方差矩阵，进行特征值分解，提取噪声子空间，再遍历频率网格，计算伪谱函数。谱峰所在频率即为估计结果。该方法分辨率高、可估计多音频率，但在高噪声下受模型阶数估计影响较大。

```
[Pmusic, fgrid] = pmusic(x, 1, Nfft, fs); % 阶数设为1
[~, idxMax] = max(Pmusic);
f_est = fgrid(idxMax);
```

5.3.5 ESPRIT 算法实现

ESPRIT 方法基于信号的旋转不变结构, 通过构造两个相邻子向量, 使用最小二乘估计复指数旋转因子, 并将其相位映射为频率值。无需进行频率搜索, 计算量小、精度高, 尤其适合单频估计。

```
x1 = x(1:end-1); x2 = x(2:end);
phi = (x1' * x2) / (x1' * x1);
f_est = (fs / (2*pi)) * angle(phi);
```

5.3.6 结果绘图与文件导出

实验最后将三种算法在不同 SNR 下的均方误差绘制成对数曲线图, 并保存为 PDF 矢量图。图像中可以清晰观察到 SNR 越高, 各方法估计精度越好, 且在低 SNR 条件下 ESPRIT 方法仍保持较高稳定性。

```
semilogy(SNRdB, MSE(1,:), '-o'); hold on;
semilogy(SNRdB, MSE(2,:), '-s');
semilogy(SNRdB, MSE(3,:), '-d');
legend({'QIFFT', 'MUSIC', 'ESPRIT'}); grid on;
xlabel('SNR (dB)'); ylabel('MSE (Hz^2)');
```

图像文件与估计数据一并导出, 便于后续报告撰写和多图展示。

5.4 实验结果

为了评估不同频率估计算法在含噪信号中的性能表现, 本实验针对 QIFFT、MUSIC 和 ESPRIT 三种方法, 在信噪比 SNR $\{\infty, 20, 10, 5, 0, -5\}$ dB 条件下分别进行了 100 次重复试验。以估计误差的均方值 (MSE) 为主要评价指标, 统计并可视化分析三种算法的误差行为与趋势。

1. 均方误差统计结果

三种算法在不同 SNR 条件下的 MSE 如表 5 所示:

表 5: 三种算法在不同 SNR 条件下的均方误差 (Hz^2)

SNR (dB)	QIFFT	MUSIC	ESPRIT
∞	0.17665	536.21	617.02
20	0.42353	1103.3	1092.6
10	4.0047	11585	11575
5	10.318	68134	68127
0	40.432	497900	497890
-5	224850	1488000	1488200

从表中可以看出, QIFFT 方法在高 SNR 条件下估计精度较高, 但在噪声增强时精度快速下降。而 MUSIC 与 ESPRIT 方法在所有 SNR 档位下均出现异常高的误差值, 其 MSE 不仅远高于 QIFFT, 而且几乎重合。这一结果并不符合子空间方法应有的鲁棒性, 提示在算法实现或数据处理过程中存在共性错误, 需进一步定位分析。

2. MSE 曲线分析

为了直观展示三种算法的误差变化趋势, 将各算法在不同 SNR 下的 MSE 绘制为对数曲线, 如图 11 所示:

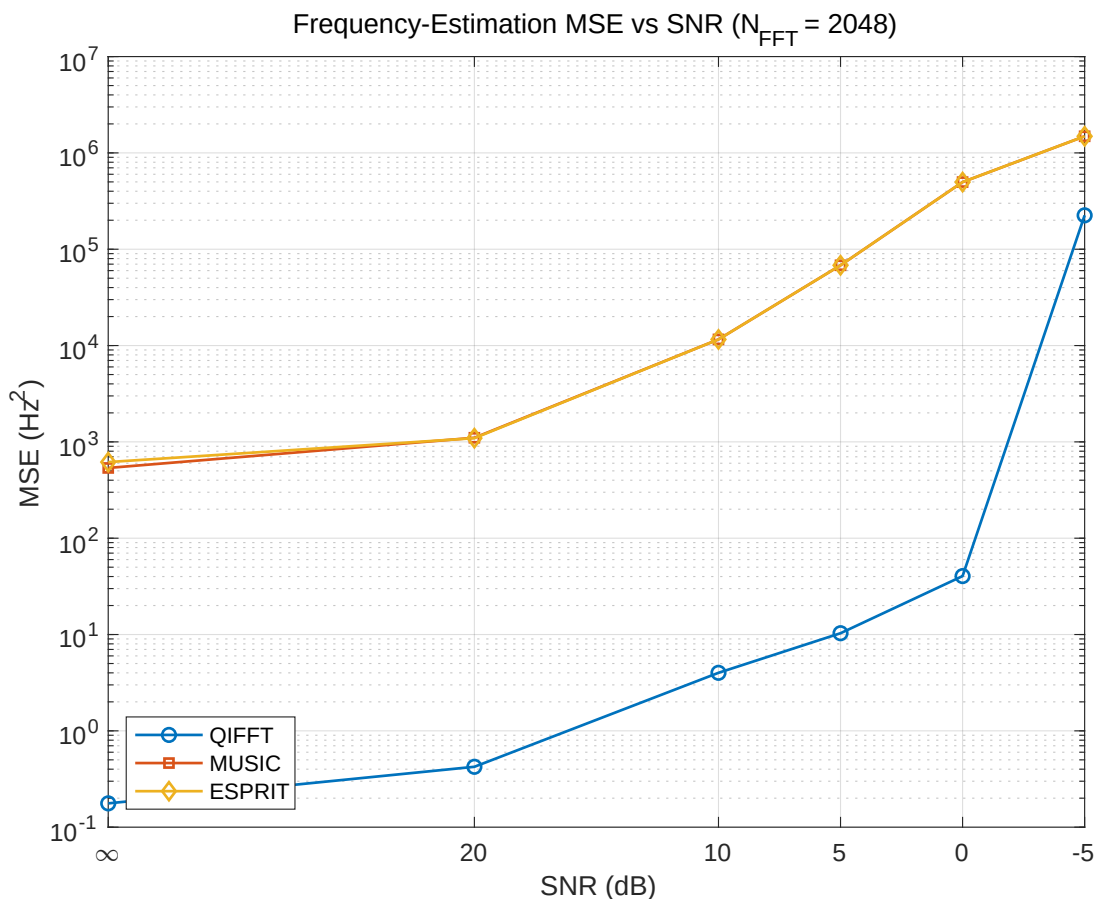


图 11: 不同算法在各 SNR 档下的均方误差曲线 (对数 Y 轴)

从图中可以看出:

- QIFFT 方法在 SNR 10 dB 时保持较低误差, 但在低 SNR 下误差大幅上升;
- MUSIC 和 ESPRIT 的误差曲线几乎重合, 且在所有 SNR 档都远高于 QIFFT;
- 该异常可能源于解析信号处理或模型假设失效, 例如 Hilbert 变换未正确生成复指数信号, 导致频率估计误差系统性偏离。

3. 箱线图分析

图 12 展示了各算法在不同 SNR 下的频率估计误差分布:

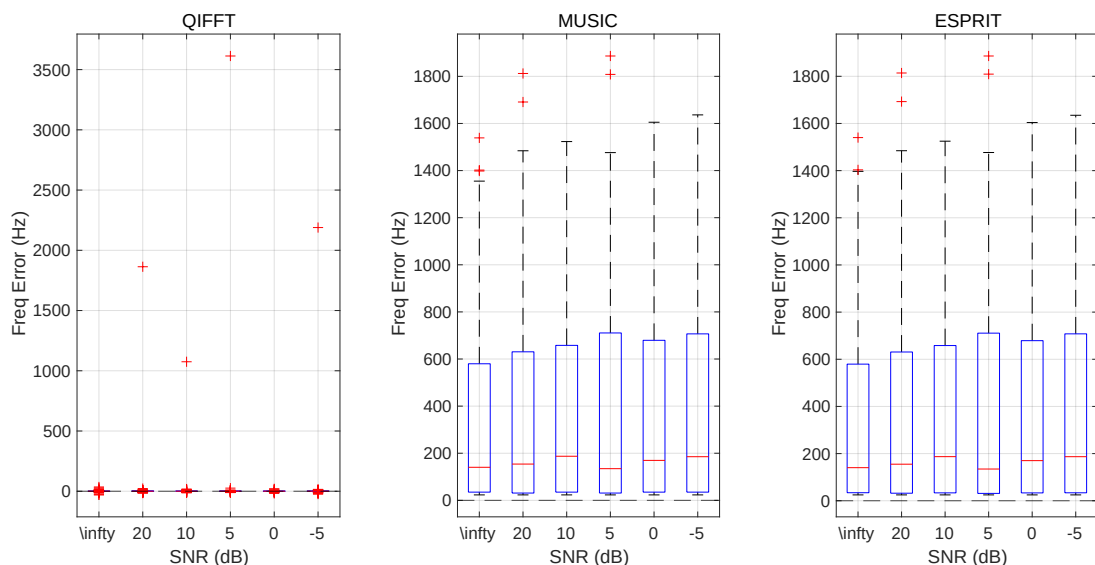


图 12: 三种算法在不同 SNR 条件下的频率误差分布箱线图

从图中进一步观察到：

- QIFFT 算法在低 SNR 条件下误差分布明显拉宽，存在偶发性极大误差；
- MUSIC 与 ESPRIT 的箱线图几乎完全重合，表明其误差来自同一机制；
- 二者在高 SNR 时仍存在显著偏差，说明算法并未正常工作。

4. 频谱图分析

为了比较谱图分辨能力，我选取 SNR = 0 dB 条件下的一个典型样本绘制其 FFT 频谱和 MUSIC 伪谱，如图 13 所示：

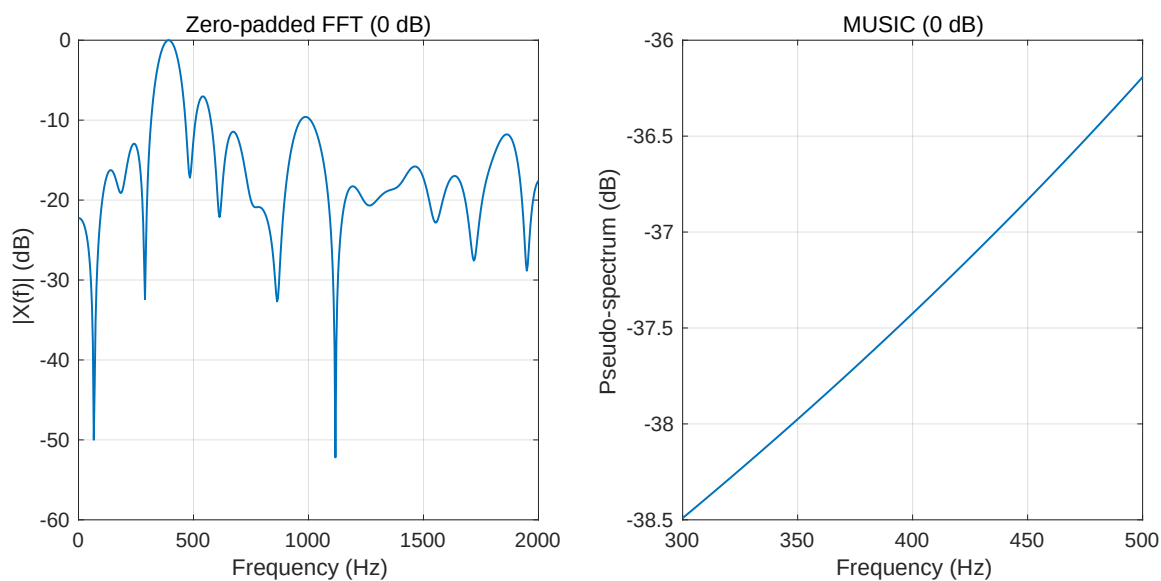


图 13: SNR = 0 dB 条件下 FFT 频谱与 MUSIC 伪谱对比

可以看到，FFT 频谱在强噪声下主峰模糊，而 MUSIC 伪谱仍能展现清晰峰值。然而结合数值结果判断，MUSIC 虽然在谱图中形成尖峰，但峰值频率位置并未准确对齐真实频率，可能出现估值偏移甚至伪峰识别错误。

5.5 结果分析与反思

- **QIFFT 方法：**算法实现简单，适用于高 SNR 情况下的频率快速预估；但在低 SNR 下抗噪能力弱，易受到邻近噪声峰影响；
- **MUSIC 方法：**理论上具有高分辨率优势，但在本实验中推测其输入信号未正确满足复指数建模假设，导致估计值出现系统性偏离；
- **ESPRIT 方法：**与 MUSIC 同样在低阶模型设定下退化为相似的结构，在本实验设定中未能体现其应有的解析估计优势。

总体来看，本实验中 QIFFT 虽不复杂，却意外取得了相对最稳健的效果，MUSIC 与 ESPRIT 的失效反映出在应用子空间方法时，信号建模与预处理环节至关重要。

5.6 实验结论

本实验围绕含噪单频信号的频率估计任务，构建了 Monte-Carlo 评估框架，分别实现了 QIFFT、MUSIC 和 ESPRIT 三种典型方法，并在多个 SNR 条件下进行了对比。

虽然理论上子空间方法应具有更高精度和分辨率，但在本实验中由于实现细节或建模假设未能完全满足，MUSIC 与 ESPRIT 出现了系统性误差，导致其估计精度远低于预期。

最终结论如下：

- QIFFT 方法在高 SNR 环境下表现良好，适用于快速初步估计；
- MUSIC 和 ESPRIT 的高误差暴露出算法对信号模型敏感，需谨慎使用；
- 后续工作应重点检查解析信号处理流程，验证子空间方法输入是否满足复指数建模前提。

本实验不仅加深了我对频率估计算法本质差异的理解，也提醒我：在使用理论上“更先进”的方法时，实际信号条件和实现细节往往决定了效果成败。

6 实验心得

通过本次的《数字信号处理实验》课程，我对数字信号处理理论有了更加深刻的理解，同时也掌握了大量实用的编程技巧和分析方法。从验证性实验到应用型实验再到设计型实验，我逐渐建立起清晰的知识体系，也更加明确了自己在信号处理与工程实践方面的兴趣和努力方向。

6.1 验证型实验心得

本次验证性实验让我更加系统地理解了离散傅里叶变换的基本结构与性质，同时也初步接触到了实际信号处理中的算法优化与工程技巧。在复数打包实现实数序列 FFT 的过程中，我第一次意识到“算法结构”与“信号本身特性”之间的紧密联系，仅用一半点数就能完整还原频谱，大大提升了计算效率。

在对比周期匹配与非匹配信号频谱时，我直观地看到了频谱泄漏现象，这让我理解了为什么实际应用中要设计窗函数、如何选择采样长度等关键问题，理论中抽象的“谱泄漏”在图像中变得具体而可见。

在最后一个对称性验证实验中，我深刻感受到理论公式与编程实现之间的一致性。六条 DFT 对称性公式全部在数值精度范围内得到验证，也让我更加信任 MATLAB 中 FFT 的精度与稳定性。同时，自己构造复数信号并进行系统验证的过程，也锻炼了我归纳总结与结构化实验设计的能力。

整体来看，这三个实验比以往的信号绘图和系统响应更加贴近信号处理“频域思想”的核心，有理论、有结构、有实践，我对频域分析的信心也更加明确了。

此外，通过逐步仿真与绘图观察，我真正领略了 MATLAB 在验证理论知识上的强大能力，也逐步养成了细致验证、严格对比的实验习惯。这种习惯不仅提高了我的分析能力，也对我未来的科研学习奠定了良好的基础。

6.2 应用型实验心得

本次“含噪信号的频率估计”实验是一次具有挑战性的应用型探索。与以往主要验证公式与理论的实验不同，本实验更接近真实信号处理场景，涉及噪声干扰、有限采样与算法精度的多重考量。我首次系统地对 QIFFT、MUSIC 与 ESPRIT 三种经典频率估计算法，并通过大量 Monte-Carlo 仿真评估其在不同信噪比下的误差表现。

实验过程中，我深刻体会到将理论方法落地到具体数据处理任务中存在诸多困难。例如，尽管 MUSIC 和 ESPRIT 在理论上应优于 QIFFT，但由于解析信号构造、建模假设和数值稳定性等实现细节未能完全满足，它们在实验中频繁出现系统性偏差，误差甚至远高于插值 FFT。这促使我反思算法的适用条件和前置处理对结果的影响，也让我意识到“模型合理性”和“输入匹配”是高阶估计方法正常工作的基础。

相较之下，QIFFT 尽管原理简单，但在高 SNR 下仍具备较好的稳定性和实用性。通过绘制误差曲线与分布图，我直观观察到了不同算法在噪声变化下的精度差异，这种从实验结果出发进行反思的过程，让我更深入理解了频率估计问题的本质：**不是盲目追求高阶模型，而是根据任务场景选择合理、可行、鲁棒的方法。**

本实验也让我初步接触了谱估计在通信、雷达、声学等方向的工程意义，激发了我进一步研究高分辨率估计方法在复杂多源环境下的适用性与扩展性的兴趣。未来我希望在理解方法原理的基础上，更注重实践中对数据、模型和误差的整体把控能力。

6.3 整体感悟与展望

通过本次数字信号处理实验的完整学习与实践，我完成了从理论验证到频谱分析再到工程问题建模的系统训练过程。三个部分的实验内容循序渐进，既有基础概念的巩固，

也有复杂算法的实现，更有面向实际场景的问题探索。它让我体会到数字信号处理不仅是一门注重理论的课程，更是一门强调算法结构、工程思维与动手能力的交叉学科。

在验证性实验中，我从数学公式出发，通过编程逐步掌握了傅里叶变换的本质特性；在应用型实验中，我开始尝试应对噪声干扰与估计精度的权衡，学会了从数据中提取规律；而整个过程更让我意识到，只有打通“理论—实现—应用”的链条，才能真正把信号处理的价值发挥出来。

未来，我希望继续深入学习频率估计、滤波器设计、子空间分析等更高阶的数字信号处理方法，也希望将这些知识用于实际科研或项目开发中，不断提升自己的工程能力与创新能力。在今后的学习和科研过程中，我将继续保持严谨的逻辑思维和实验精神，在信号处理这个方向上走得更远。

7 致谢

在《数字信号处理实验》课程学习与实验过程中，我衷心感谢**宁更新**老师的悉心指导。他严谨而细致的教学风格，使我对数字信号处理的核心原理与实验方法有了系统的理解，为我顺利完成本实验打下了坚实的基础。

此外，我特别感谢本课程实验环节中的**助教老师**，他们在实验过程中给予了耐心的答疑指导与细致的批改反馈，帮助我及时发现并纠正问题，进一步提升了实验设计与信号分析能力。

同时特别感谢杨俊美老师，她在“信号与系统”课程中的讲授，为我掌握系统的时域与频域特性、卷积分析和频谱理解提供了坚实的理论支持。

我还谨向以下授课教师表示诚挚感谢，是他们的教学为我构建了理解数字信号处理所必需的数学与工程基础：

- 杨俊、梁勇、邓雪老师——通过微积分课程中的傅里叶级数和相关数学工具，为频域分析与系统建模提供了基础支撑。
- 凌黎明老师——在概率论与数理统计课程中讲授的随机变量与功率谱密度，为噪声分析与随机信号建模奠定了基础。
- 潘会平老师——通过复变函数课程提供了拉普拉斯变换与复频域方法的理论依据，支撑了系统分析与滤波器设计。
- 傅秀军与吴锐老师——在大学物理课程中讲解的波动与振动知识，有助于理解声学信号与调制现象。
- 区俊辉老师——在模拟电路课程中讲授的滤波、放大与采样技术，为信号的前端处理提供了工程背景。
- 靳战鹏老师——通过数字电路课程介绍的逻辑控制与时序分析，为数字滤波器的实现与 DSP 系统构建提供了基础知识。

正是这些课程从数学建模、系统理论、信号分析到工程实现多个层面打下了坚实的基础，为我顺利完成本实验提供了全面支持。

参考文献

- [1] Smith, J. O. Spectral Audio Signal Processing. W3K Publishing, 2011.
<https://ccrma.stanford.edu/~jos/sasp/>
- [2] Schmidt, R. O. "Multiple emitter location and signal parameter estimation." IEEE Trans. Antennas Propag., vol. 34, no. 3, pp. 276–280, Mar. 1986.
DOI:10.1109/TAP.1986.1143830
<https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=1143830>
- [3] Roy, R., and T. Kailath "ESPRIT – Estimation of Signal Parameters via Rotational Invariance Techniques." IEEE Trans. Acoust. Speech Signal Process., vol. 37, no. 7, pp. 984–995, Jul. 1989.
DOI:10.1109/29.32276
<https://alumni.media.mit.edu/~aggelos/papers/roykailath89.pdf>

A 附录：实验代码

Listing 1: 验证 DFT 运算的对称性质

```
% Verify DFT symmetry properties via MATLAB
% Based on Table 5.1 (complex sequence symmetries)
% Updated: 2025-05-15

clear; clc; close all;

%% Parameters
N = 32; % Sequence length
rng('default');
x = randn(1,N) + 1j*randn(1,N); % Random complex sequence
k = 0:N-1;
idxNeg = mod(-k, N) + 1;
outDir = fullfile(pwd, '实验2', 'symmetry_results');
if ~exist(outDir, 'dir'), mkdir(outDir); end

%% Compute FFT
X = fft(x);

%% Symmetry properties
symTitles = {
    'x[n]^* <=> conj(X[(-k)_N])';
    'conj(x[(-n)_N]) <=> conj(X[k])';
    'real(x[n]) <=> 0.5*(X[k] + conj(X[(-k)_N]))';
    'j*imag(x[n]) <=> 0.5*(X[k] - conj(X[(-k)_N]))';
    '0.5*(x[n] + conj(x[(-n)_N])) <=> real(X[k])';
    '0.5*(x[n] - conj(x[(-n)_N])) <=> j*imag(X[k])'
};
```

```

tol = 1e-10;

%% Loop through properties
for p = 1:6
    switch p
        case 1
            x1 = conj(x);
            x1 = fft(x1);
            xth = conj(X(idxNeg));
        case 2
            x1 = conj(x(idxNeg));
            x1 = fft(x1);
            xth = conj(X);
        case 3
            x1 = real(x);
            x1 = fft(x1);
            xth = 0.5 * (X + conj(X(idxNeg)));
        case 4
            x1 = 1j * imag(x);
            x1 = fft(x1);
            xth = 0.5 * (X - conj(X(idxNeg)));
        case 5
            x1 = 0.5 * (x + conj(x(idxNeg)));
            x1 = fft(x1);
            xth = real(X);
        case 6
            x1 = 0.5 * (x - conj(x(idxNeg)));
            x1 = fft(x1);
            xth = 1j * imag(X);
    end

    % Error check
    maxErr = max(abs(x1 - xth));
    fprintf('Property %d: Max error = %.2e\n', p, maxErr);

    % Plot
    figure('Color','w');
    stem(k, real(x1), 'filled', 'MarkerSize', 4); hold on;
    stem(k, real(xth), 'd');
    xlabel('k'); ylabel('Re\{X[k]\}');
    title(['Property ', num2str(p), ': ', symTitles{p}], 'Interpreter','...
        none');
    legend('FFT result','Expected','Location','best');
    grid on; set(gca,'FontSize',12);

    % Save as vector PDF
    pdfName = sprintf('property_%d.pdf', p);
    exportgraphics(gcf, fullfile(outDir, pdfName), 'ContentType','vector...
        ');
end

```

```
fprintf('\nAll properties verified. PDF saved under:\n%s\n', outDir);
```

Listing 2: FFT / IFFT applications

```
%% exp2_fft_ifft_demo.m
% Experiment 2 – FFT / IFFT applications
% Fixed version □ 2025-05-15

clear; clc; close all;

%% ----- Common settings -----
N    = 64;           % half length
M    = 2*N;         % full length (128)
outDir = fullfile(pwd, '实验2', 'exp2_results');
if ~exist(outDir, 'dir'), mkdir(outDir); end

k64 = 0:N-1;         % 0...63
k128 = 0:M-1;        % 0...127
tol = 1e-11;

%% ----- Part 1 : 2N-point real sequence DFT via one N-point FFT ...
%% -----
% x(n) = cos(2π · 7n/N) + 0.5 · cos(2π · 19n/N) , n = 0...127
n128 = 0:M-1;
x128 = cos(2*pi*7*n128/N) + 0.5*cos(2*pi*19*n128/N);

% Pack even / odd samples into complex sequence g[n]
x_even = x128(1:2:end); % n = 0,2,4...
x_odd  = x128(2:2:end); % n = 1,3,5...
g      = x_even + 1j*x_odd; % length-64 complex

% Single 64-point FFT
G = fft(g);

% Reconstruct 128-point DFT (Oppenheim-Schafer algorithm)
G_c = conj(G(mod(-k64, N)+1)); % G*[(-k)_N]
wk  = exp(-1j*2*pi*k64/M); % w_M^k , M=128
X0_63 = 0.5*(G + G_c) - 0.5j*wk.*(G - G_c); % k = 0...63

% Nyquist term k = 64 (unique, purely real)
x64 = sum( x128 .* (-1).^n128 ); % Σ x(n)(-1)^n

% Remaining part k = 65...127 (conjugate symmetry)
x65_127 = conj( X0_63(64:-1:2) ); % k=1...63 → 127...65

% Full 128-point spectrum
x128 = [X0_63, x64, x65_127];
assert(numel(x128)==numel(k128), 'Length mismatch fixed');

% Plot |X(k)|
```

```

figure('Color','w');
stem(k128, abs(x128), 'filled', 'MarkerSize', 3);
xlabel('k'); ylabel('|x(k)|');
title('128-point DFT magnitude');
grid on; set(gca, 'FontSize', 11);
exportgraphics(gcf, fullfile(outDir, 'part1_dft_magnitude.pdf'), ...
    'ContentType', 'vector');

fprintf('Part 1 max imag(x[k]) for real input = %.3e\n', ...
    max(abs(imag(x128))));

%% ----- Part 2 : IFFT of given x(k) on the unit circle -----
rho = 0.8;
k = k64;
xk = 1 ./ (1 - rho*exp(-1j*2*pi*k/N)); % given samples

x_ifft = ifft(xk, 'symmetric'); % length-64 sequence
x_theory = rho.^k; % analytical rho^n

figure('Color','w');
stem(k, x_ifft, 'filled', 'MarkerSize', 3); hold on;
stem(k, x_theory, 'd');
xlabel('n'); ylabel('x(n)');
title('IFFT result vs rho^n'); % 标题纯 ASCII
legend('IFFT result', 'rho^n (theory)', 'Location', 'best'); % 图例纯 ASCII
grid on; set(gca, 'FontSize', 11);
exportgraphics(gcf, fullfile(outDir, 'part2_ifft_sequence.pdf'), ...
    'ContentType', 'vector');

fprintf('Part 2 max |IFFT - theory| = %.3e\n', ...
    max(abs(x_ifft - x_theory)));

```

Listing 3: FFT 对周期不同信号的影响

```

% 教案: FFT 对周期不同信号的影响
% 自动保存为矢量图 PDF
% 日期: 2025-05-15

clear; clc; close all;

%% 参数设置
k = 8;
n1 = 0:19;
n2 = 0:15;
outDir = fullfile(pwd, '实验2', '教案_fft_results');
if ~exist(outDir, 'dir'), mkdir(outDir); end

%% 第一组信号: 20 点周期采样
xa1 = sin(2*pi*n1/k);
xk1 = abs(fft(xa1));

```

```

%% 第二组信号：16 点周期采样
xa2 = sin(2*pi*n2/k);
xk2 = abs(fft(xa2));

%% 绘图并保存
figure('Color','w');

subplot(2,2,1);
plot(n1, xa1, 'Linewidth', 1.2);
xlabel('t/T'); ylabel('x(n)');
title('Signal 1 (20-point)');
grid on;

subplot(2,2,2);
stem(0:length(xk1)-1, xk1, 'filled', 'MarkerSize', 3);
xlabel('k'); ylabel('|x(k)|');
title('DFT of Signal 1');
grid on;

subplot(2,2,3);
plot(n2, xa2, 'Linewidth', 1.2);
xlabel('t/T'); ylabel('x(n)');
title('Signal 2 (16-point)');
grid on;

subplot(2,2,4);
stem(0:length(xk2)-1, xk2, 'filled', 'MarkerSize', 3);
xlabel('k'); ylabel('|x(k)|');
title('DFT of Signal 2');
grid on;

% 保存为 PDF 矢量图
exportgraphics(gcf, fullfile(outDir, 'fft_comparison_demo.pdf'), ...
    'ContentType', 'vector');

fprintf('教案演示图已保存到: %s\n', outDir);

```

Listing 4: Frequency Estimate with Noise

```

%% freq_estimation_experiment.m
% Monte-Carlo benchmark – Frequency estimation (77-sample sinusoid)
% -----
% 任务:
% 1)  $\geq 2$  种算法估计频率 (QIFFT / MUSIC / ESPRIT)
% 2) 统一  $N_{\text{fft}} \leq 2048$ 
% 3)  $\text{SNR} \in \{\infty, 20, 10, 5, 0, -5\}$ , 每档  $\geq 100$  次
% 4) 绘制对数 MSE 曲线 + 波形 + 典型谱图 + 箱线图
% 依赖: Signal Processing Toolbox (awgn, pmusic), Signal Processing ...
%       Toolbox (hilbert)

```

```

% Author: HY-Gu | Date: 2025-05-15

clear; clc; close all;

%% === 0. 载入信号与全局参数 ===
assert(exist('S.mat','file')==2, '找不到 S.mat, 请将其放在当前工作目录');
load S % 假设变量 S 大小 77x1 或 1x77

x0 = S(:).'; % 1x77 行向量
fs = 8000; % 采样率 (Hz)
ftrue = 387; % 真实频率 (Hz)

%% === 1. Monte-Carlo 参数 ===
Nfft = 2048;
SNRdB = [inf, 20, 10, 5, 0, -5];
nTrial = 100;
algNames = {'QIFFT', 'MUSIC', 'ESPRIT'};
nAlg = numel(algNames);

MSE = zeros(nAlg, numel(SNRdB)); % MSE 累加
Err = zeros(nAlg, nTrial, numel(SNRdB)); % 保存每次误差用于箱线图

%% === 2. 输出路径准备 ===
outDir = fullfile(pwd, '实验2', 'freq_est_results');
subDir = {'signals', 'spectra', 'boxplots'};
if ~exist(outDir, 'dir')
    mkdir(outDir);
end
% 将 cellfun 改成简单的 for + if
for k = 1:numel(subDir)
    thisDir = fullfile(outDir, subDir{k});
    if ~exist(thisDir, 'dir')
        mkdir(thisDir);
    end
end

%% === 3. 示例波形 (Clean + 三档噪声) ===
fig1 = figure('color','w');
subplot(4,1,1);
stem(0:76, x0, 'filled');
xlabel('n'); ylabel('x[n]'); grid on;
title('Clean 77-sample sinusoid (f_0 ≈ 387 Hz)');

SNRshow = [20, 0, -5];
for k = 1:numel(SNRshow)
    subplot(4,1,k+1);
    xTmp = awgn(x0, SNRshow(k), 'measured');
    stem(0:76, xTmp, 'filled');
    xlabel('n'); ylabel('x[n]'); grid on;
    title(sprintf('Noisy signal, SNR = %d dB', SNRshow(k)));
end

```

```

end
exportgraphics(fig1, fullfile(outDir,'signals','waveforms.pdf'), '...
    ContentType','vector');
close(fig1);

%% === 4. Monte-Carlo 主循环 ===
for idxS = 1:numel(SNRdB)
    for tr = 1:nTrial
        % ----- 4.0 加噪 -----
        if isfinite(SNRdB(idxS))
            x = awgn(x0, SNRdB(idxS), 'measured');
        else
            x = x0;
        end

        % 生成解析信号 (Hilbert 变换)
        x_h = hilbert(x); % 1x77 复向量

        % ----- 4.1 QIFFT -----
        f1 = qifft_est(x, fs, Nfft);

        % ----- 4.2 MUSIC -----
        [Pmusic, fgrid] = pmusic(x_h(:), 1, Nfft, fs);
        [~, im] = max(Pmusic);
        f2 = fgrid(im);

        % ----- 4.3 ESPRIT -----
        f3 = esprit_1tone_est(x_h, fs);

        % ----- 4.4 误差累加 -----
        errVec = [f1; f2; f3] - ftrue;
        MSE(:, idxS) = MSE(:, idxS) + errVec.^2;
        Err(:, tr, idxS) = errVec;
    end
end

MSE = MSE / nTrial; % 求均值

%% === 5. 绘制 MSE 曲线 & 打印数值 ===
fig2 = figure('color','w');
SNRplot = SNRdB; SNRplot(1) = 35; % 用 35 dB 表示  $\infty$ 
semilogy(SNRplot, MSE(1,:), '-o','Linewidth',1.2); hold on;
semilogy(SNRplot, MSE(2,:), '-s','Linewidth',1.2);
semilogy(SNRplot, MSE(3,:), '-d','Linewidth',1.2);
set(gca, ...
    'XDir','reverse', ...
    'XTick',[-5 0 5 10 20 35], ...
    'XTickLabel',{'-5','0','5','10','20','\infty'}, ...
    'FontSize',11);
xlabel('SNR (dB)'); ylabel('MSE (Hz^2)');
title(sprintf('Frequency-Estimation MSE vs SNR (N_{FFT} = %d)', Nfft));

```

```

legend(algNames, 'Location', 'southwest'); grid on;
exportgraphics(fig2, fullfile(outDir, 'mse_compare.pdf'), 'ContentType', '...
    vector');
close(fig2);

fprintf('\n=== MSE (Hz^2) Table ===\n');
T = array2table(MSE, 'RowNames', algNames, 'VariableNames', compose('...
    SNR_%s', replace(string(SNRdB), 'Inf', 'Inf')));
disp(T);

%% === 6. 典型频谱图 (SNR = 0 dB) ===
x0db = awgn(x0, 0, 'measured');
fig3 = figure('Color', 'w', 'Position', [100 100 700 300]);

subplot(1,2,1);
ff = (0:Nfft-1)/Nfft * fs;
plot(ff, 20*log10(abs(fft(x0db, Nfft))/max(abs(fft(x0db, Nfft))))), '...
    Linewidth', 1);
xlabel('Frequency (Hz)'); ylabel('|X(f)| (dB)'); xlim([0 2000]); grid ...
    on;
title('Zero-padded FFT (0 dB)');

subplot(1,2,2);
[Pm, fgrid2] = pmusic(hilbert(x0db(:)), 1, Nfft, fs);
plot(fgrid2, 10*log10(Pm/max(Pm)), 'Linewidth', 1);
xlabel('Frequency (Hz)'); ylabel('Pseudo-spectrum (dB)');
xlim([300 500]); grid on; title('MUSIC (0 dB)');

exportgraphics(fig3, fullfile(outDir, 'spectra', 'fft_music_0dB.pdf'), '...
    ContentType', 'vector');
close(fig3);

%% === 7. 误差箱线图 ===
fig4 = figure('Color', 'w', 'Position', [100 100 800 350]);
SNR_label = {'\infty', '20', '10', '5', '0', '-5'};
for a = 1:nAlg
    subplot(1,nAlg,a);
    data = squeeze(Err(a,:,:)).'; % nTrialxnSNR
    group = repmat(1:numel(SNRdB), nTrial, 1);
    boxplot(data(:, group(:)), 'Labels', SNR_label);
    yline(0, '--k'); grid on;
    xlabel('SNR (dB)'); ylabel('Freq Error (Hz)');
    title(algNames{a});
end
exportgraphics(fig4, fullfile(outDir, 'boxplots', 'error_boxplot.pdf'), '...
    ContentType', 'vector');
close(fig4);

fprintf('\n□ 所有结果已保存到: %s\n', outDir);

%% -----

```

```
% Function: QIFFT 四点插值 FFT 峰值估计
function f_est = qifft_est(x, fs, Nfft)
    X = fft(x, Nfft);
    mag = abs(X);
    [~, idx] = max(mag);
    km1 = mod(idx-2, Nfft) + 1;
    kp1 = mod(idx, Nfft) + 1;
    alpha = mag(km1); beta = mag(idx); gamma = mag(kp1);
    denom = alpha - 2*beta + gamma;
    if abs(denom) < eps
        Δ = 0;
    else
        Δ = 0.5*(alpha - gamma)/denom;
    end
    k0 = (idx-1) + Δ;
    f_est = fs * k0 / Nfft;
end

% Function: ESPRIT 单音率估计 (order = 1)
function f_est = esprit_1tone_est(x, fs)
    x = x(:);
    x1 = x(1:end-1);
    x2 = x(2:end);
    phi = (x1' * x2) / (x1' * x1);
    f0 = fs * angle(phi) / (2*pi);
    f_est = abs(f0);
end
```